

PROJETO INTEGRADOR — BLOCO 4

Desenvolvimento | Checkpoint 2

Disciplina:	Projeto Integrador
Entrega:	Bloco 4 — Desenvolvimento + Checkpoint 2
Projeto:	StudyPath — Plataforma SaaS de Gerenciamento de Estudos para Vestibular

1. IDENTIFICAÇÃO DO GRUPO

Nome Completo	RA
Cristian Silva Rodrigues	924115784
Kayky Henrique da Silva	924112141

2. PADRÕES DE PROJETO (GoF)

Foram identificados e aplicados 4 padrões de projeto GoF no StudyPath, cobrindo as categorias Criacional, Estrutural e Comportamental.

Padrão 1 — Factory Method (Criacional)

Atributo	Descrição
Categoria	Criacional
Problema resolvido	O sistema precisa gerar diferentes tipos de notificações (e-mail, push, SMS) sem que o código cliente conheça as classes concretas. Adicionar um novo canal exigiria alteração em vários pontos do código.
Solução aplicada	A interface NotificationFactory define o método create(). Cada canal (EmailFactory, PushFactory) implementa a criação do seu tipo específico de notificação, isolando a lógica de instanciação.
Onde é usado	Notification Service — ao enfileirar mensagens no Redis, o serviço chama a factory correta conforme o tipo definido no payload da fila.

Diagrama UML — Factory Method:

<>		EmailNotification
NotificationFactory	■■■■■■■■■■>	PushNotification
+ create(): Notification		SMSNotification

Implementação (TypeScript):

```
// interfaces/notification.factory.ts
interface Notification {
  send(to: string, message: string): Promise<void>;
}

interface NotificationFactory {
  create(): Notification;
}

// factories/email.factory.ts
```

```

class EmailNotification implements Notification {
  async send(to: string, message: string) {
    await sendgrid.send({ to, subject: "StudyPath", text: message });
  }
}

class EmailFactory implements NotificationFactory {
  create(): Notification { return new EmailNotification(); }
}

// factories/push.factory.ts
class PushNotification implements Notification {
  async send(to: string, message: string) {
    await fcm.send({ token: to, notification: { body: message } });
  }
}

class PushFactory implements NotificationFactory {
  create(): Notification { return new PushNotification(); }
}

// notification.service.ts – uso da factory
const factories: Record<string, NotificationFactory> = {
  email: new EmailFactory(),
  push: new PushFactory(),
};

async function notify(type: string, to: string, msg: string) {
  const factory = factories[type]; // sem if/switch
  await factory.create().send(to, msg);
}

```

Padrão 2 — Strategy (Comportamental)

Atributo	Descrição
Categoria	Comportamental
Problema resolvido	O algoritmo de repetição espaçada pode variar (SM-2, Leitner, intervalo fixo). Hardcodar a lógica no serviço tornaria difícil trocar ou adicionar algoritmos sem risco de regressão.
Solução aplicada	A interface <code>SpacedRepetitionStrategy</code> define o método <code>calculateNextInterval()</code> . Cada algoritmo (<code>SM2Strategy</code> , <code>LeitnerStrategy</code>) implementa sua própria lógica, sendo injetada no <code>FlashcardService</code> .
Onde é usado	<code>Study Service</code> — <code>FlashcardService</code> recebe a estratégia via injeção de dependência, permitindo trocar o algoritmo sem alterar o serviço.

Implementação (TypeScript):

```

// strategies/spaced-repetition.strategy.ts
interface ReviewResult { grade: number; repetitions: number; easeFactor: number; }

interface SpacedRepetitionStrategy {

```

```

    calculateNextInterval(result: ReviewResult): number; // retorna dias
}

// strategies/sm2.strategy.ts
class SM2Strategy implements SpacedRepetitionStrategy {
    calculateNextInterval({ grade, repetitions, easeFactor }: ReviewResult): number {
        if (grade < 3) return 1; // repetir amanha
        const ef = Math.max(1.3, easeFactor + 0.1 - (5 - grade) * 0.08);
        if (repetitions === 0) return 1;
        if (repetitions === 1) return 6;
        return Math.round(6 * Math.pow(ef, repetitions - 1));
    }
}

// strategies/leitner.strategy.ts
class LeitnerStrategy implements SpacedRepetitionStrategy {
    private boxes = [1, 3, 7, 14, 30];
    calculateNextInterval({ grade, repetitions }: ReviewResult): number {
        const box = grade >= 4
            ? Math.min(repetitions + 1, this.boxes.length - 1)
            : 0;
        return this.boxes[box];
    }
}

// services/flashcard.service.ts
class FlashcardService {
    constructor(private strategy: SpacedRepetitionStrategy) {}

    async review(cardId: string, grade: number) {
        const card = await FlashcardRepository.findById(cardId);
        const days = this.strategy.calculateNextInterval({
            grade, repetitions: card.sm2.repetitions,
            easeFactor: card.sm2.easeFactor,
        });
        await FlashcardRepository.updateNextReview(cardId, days);
    }
}

// Injeção – pode trocar sem mudar FlashcardService
const service = new FlashcardService(new SM2Strategy());

```

Padrão 3 — Observer (Comportamental)

Atributo	Descrição
Categoria	Comportamental
Problema resolvido	Quando o aluno conclui uma sessão de estudo, vários subsistemas precisam ser notificados: atualizar streak, publicar evento de analytics, acionar notificação de parabéns. Chamar cada um diretamente gera acoplamento forte.

Atributo	Descrição
Solução aplicada	O EventEmitter do Node.js implementa o padrão Observer nativamente. O Study Service emite o evento 'session.completed' e os observers (StreakObserver, AnalyticsObserver, NotificationObserver) reagem de forma desacoplada.
Onde é usado	Study Service — ao finalizar uma sessão de estudo, o evento é emitido e os observers registrados executam suas lógicas independentemente.

Implementação (TypeScript):

```
// events/study-events.ts
import { EventEmitter } from 'events';
export const studyBus = new EventEmitter();

// observers/streak.observer.ts
studyBus.on('session.completed', async ({ userId, date }) => {
  const streak = await RedisPublisher.getStreak(userId);
  const newStreak = isConsecutiveDay(streak.lastStudy, date)
    ? streak.days + 1 : 1;
  await RedisPublisher.setStreak(userId, newStreak, date);
  console.log(`Streak atualizado: ${newStreak} dias`);
});

// observers/analytics.observer.ts
studyBus.on('session.completed', async ({ userId, sessionId, minutes }) => {
  await AnalyticsClient.track({
    event: 'study_session_completed',
    userId, sessionId,
    properties: { duration_minutes: minutes },
  });
});

// observers/notification.observer.ts
studyBus.on('session.completed', async ({ userId, streak }) => {
  if (streak % 7 === 0) { // marcos de 7 dias
    await NotificationFactory.create('push').send(
      userId, `Parabens! ${streak} dias de estudo seguidos!`
    );
  }
});

// schedule.service.ts – emissao do evento
async function completeSession(sessionId: string, userId: string) {
  const session = await ScheduleRepository.complete(sessionId);
  studyBus.emit('session.completed', { // observers reagem aqui
    userId, sessionId,
    minutes: session.durationMinutes,
    date: new Date(),
  });
}
```

Padrão 4 — Decorator (Estrutural)

Atributo	Descrição
Categoria	Estrutural
Problema resolvido	Usuários do plano Básico têm acesso limitado a funcionalidades. Adicionar verificações de plano com if/else em cada endpoint polui o código e dificulta manutenção.
Solução aplicada	Decorators TypeScript/NestJS envolvem os métodos do controller, verificando o plano do usuário antes de executar a lógica. <code>@RequiresPlan('premium')</code> bloqueia chamadas de usuários do plano gratuito com status 403.
Onde é usado	API Gateway / Controllers — endpoints de simulados ilimitados, relatório PDF e filtros avançados por banca são decorados com <code>@RequiresPlan</code> .

Implementação (TypeScript + NestJS):

```
// decorators/requires-plan.decorator.ts
import { SetMetadata } from '@nestjs/common';

export const RequiresPlan = (plan: 'premium' | 'institutional') =>
  SetMetadata('required_plan', plan);

// guards/plan.guard.ts
@Injectable()
class PlanGuard implements CanActivate {
  canActivate(ctx: ExecutionContext): boolean {
    const requiredPlan = this.reflector.get('required_plan', ctx.getHandler());
    if (!requiredPlan) return true; // sem restricao

    const { user } = ctx.switchToHttp().getRequest();
    const planHierarchy = { free: 0, premium: 1, institutional: 2 };

    if (planHierarchy[user.plan] < planHierarchy[requiredPlan]) {
      throw new ForbiddenException(
        `Esta funcionalidade requer o plano ${requiredPlan}.`
      );
    }

    return true;
  }
}

// controllers/question.controller.ts
@Controller('questions')
@UseGuards(AuthGuard, PlanGuard)
class QuestionController {
  @Get('simulate')
  @RequiresPlan('premium') // so premium pode acessar
  async runSimulation(@Query() dto: SimulationDto) {
    return this.questionService.generateSimulation(dto);
  }
}
```

```
@Get('filter/advanced')
@RequiresPlan('premium')
async filterAdvanced(@Query() dto: AdvancedFilterDto) {
    return this.questionService.filterAdvanced(dto);
}
}
```

3. DOCUMENTAÇÃO DE APIs — OPENAPI / SWAGGER

A documentação completa das APIs do StudyPath foi gerada com **Swagger / OpenAPI 3.0** integrado ao NestJS. O link da documentação interativa está disponível abaixo:

<https://github.com/mindrodr/studypath-vestibular/blob/main/docs/api/openapi.yaml>

3.1 Endpoints Documentados

Método	Endpoint	Descrição	Auth	Plano
POST	/auth/register	Cadastro de novo usuário com e-mail e senha	Não	Todos
POST	/auth/login	Login com e-mail/senha. Retorna JWT + refresh token	Não	Todos
POST	/auth/refresh	Renova o access token usando refresh token	Não	Todos
POST	/auth/google	Login via Google OAuth 2.0	Não	Todos
GET	/schedules/current	Retorna o cronograma da semana atual do usuário	JWT	Todos
POST	/schedules/generate	Gera novo cronograma com base no diagnóstico e metas	JWT	Todos
PATCH	/schedules/sessions/{id}	Marca sessão como concluída e dispara evento Observer	JWT	Todos
GET	/questions	Lista questões com filtro por matéria, banca e dificuldade	JWT	Todos
POST	/questions/{id}/answer	Registra resposta do aluno e retorna gabarito + explicação	JWT	Todos
GET	/questions/simulate	Gera simulado com N questões aleatórias por banca	JWT	Premium
GET	/flashcards/due	Lista cards com revisão pendente para hoje (SM-2)	JWT	Todos
POST	/flashcards	Cria novo flashcard para o usuário	JWT	Todos
POST	/flashcards/{id}/review	Registra resultado da revisão (grade 0-5) e agenda próxima	JWT	Todos
GET	/analytics/performance	Retorna métricas de desempenho por matéria e período	JWT	Todos

Método	Endpoint	Descrição	Auth	Plano
GET	/analytics/report	Gera e retorna relatório PDF de desempenho mensal	JWT	Premium
GET	/topics/progress	Retorna mapa de progresso por matéria e tópico	JWT	Todos
PATCH	/topics/{id}/status	Atualiza status do tópico (pending/reviewing/mastered)	JWT	Todos
POST	/ai/recommend	Retorna recomendações de estudo personalizadas via IA	JWT	Premium
POST	/ai/explain	Solicita explicação detalhada de uma questão errada via GPT	JWT	Premium
GET	/users/me	Retorna perfil completo do usuário autenticado	JWT	Todos
PATCH	/users/me	Atualiza dados do perfil (nome, meta, data da prova)	JWT	Todos

3.2 Exemplo de Request e Response — POST /questions/{id}/answer

```
// REQUEST
POST /questions/uuid-da-questao/answer
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "chosen": "c"
}

// RESPONSE 200 OK
{
  "question_id": "uuid-da-questao",
  "chosen": "c",
  "is_correct": true,
  "correct_answer": "c",
  "explanation": "f(2) = 2(2)^2 - 3(2) + 1 = 8 - 6 + 1 = 3",
  "performance": {
    "subject": "Matematica",
    "correct_total": 28,
    "total_answers": 35,
    "accuracy": 0.80
  }
}

// RESPONSE 401 Unauthorized
{ "statusCode": 401, "message": "Token invalido ou expirado" }
```

```
// RESPONSE 403 Forbidden
{ "statusCode": 403, "message": "Esta funcionalidade requer o plano premium." }

// RESPONSE 404 Not Found
{ "statusCode": 404, "message": "Questao nao encontrada" }
```

3.3 Exemplo de Request e Response — GET /flashcards/duo

```
// REQUEST

GET /flashcards/duo?limit=10&subject=Biologia
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

// RESPONSE 200 OK
{
  "total_duo": 4,
  "cards": [
    {
      "_id": "64a1b2c3d4e5f6789abc0001",
      "subject": "Biologia",
      "topic": "Fotossintese",
      "front": "O que e fotossintese?",
      "sm2": {
        "interval_days": 7,
        "next_review": "2026-06-23",
        "last_grade": 4
      }
    },
    {
      "_id": "64a1b2c3d4e5f6789abc0002",
      "subject": "Biologia",
      "topic": "Mitose",
      "front": "Quais as fases da mitose?",
      "sm2": {
        "interval_days": 1,
        "next_review": "2026-06-23",
        "last_grade": 2
      }
    }
  ]
}
```

4. INTELIGÊNCIA ARTIFICIAL NA APLICAÇÃO

O StudyPath integra IA em três funcionalidades principais, utilizando a API da OpenAI (GPT-4o) e modelos de recomendação próprios para personalizar a experiência de estudo.

4.1 Funcionalidade 1 — Recomendação Inteligente de Estudo

Atributo	Descrição
Problema	O aluno não sabe quais matérias priorizar dado seu desempenho atual e o tempo disponível até a prova.
Solução com IA	Um modelo de recomendação analisa o histórico de desempenho por matéria, o gap em relação à meta, os dias até a prova e o streak atual. Com esses dados, a IA gera um plano de estudo personalizado para os próximos 7 dias.
API utilizada	OpenAI GPT-4o-mini via API REST. Escolhida pela qualidade de raciocínio e custo-benefício para prompts de análise educacional.
Endpoint	POST /ai/recommend — recebe perfil e histórico do aluno, retorna recomendações em JSON estruturado.

Prova de Conceito (PoC) — Código Node.js:

```
// services/ai-recommendation.service.ts
import OpenAI from 'openai';

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

interface StudentProfile {
  name: string;
  target_exam: string;
  days_to_exam: number;
  performance: { subject: string; accuracy: number }[];
  available_hours_per_week: number;
}

async function generateStudyRecommendation(profile: StudentProfile) {
  const prompt = `
Voce e um tutor especializado em vestibulares brasileiros.
Analise o perfil do aluno e crie um plano de estudo para 7 dias.

PERFIL DO ALUNO:
- Nome: ${profile.name}
- Prova alvo: ${profile.target_exam}
- Dias ate a prova: ${profile.days_to_exam}
- Horas disponiveis por semana: ${profile.available_hours_per_week}h

DESEMPENHO POR MATERIA:
${profile.performance.map(p =>
  "- " + p.subject + ": " + Math.round(p.accuracy * 100) + "% de acerto"
)}.join("\n")`
```

Retorne APENAS um JSON com a estrutura:

```
{
  "priority_subjects": ["Materia1", "Materia2"],
  "weekly_plan": [
    { "day": "Segunda", "subject": "Matematica", "hours": 2, "focus": "Funcoes" }
  ],
  "tips": ["Dica 1", "Dica 2"]
}`;

const response = await openai.chat.completions.create({
  model: 'gpt-4o-mini',
  messages: [{ role: 'user', content: prompt }],
  response_format: { type: 'json_object' },
  temperature: 0.3,
});

return JSON.parse(response.choices[0].message.content);
}

// Resultado esperado:
// {
//   "priority_subjects": ["Historia", "Fisica"],
//   "weekly_plan": [
//     { "day": "Segunda", "subject": "Historia", "hours": 2,
//       "focus": "Brasil Republica" },
//     { "day": "Terca", "subject": "Fisica", "hours": 1.5,
//       "focus": "Cinematica" }
//   ],
//   "tips": [
//     "Foque nas materias abaixo de 60% de acerto",
//     "Revise fisica antes das questoes de exatas"
//   ]
// }
```

4.2 Funcionalidade 2 — Explicação de Questões Erradas

Atributo	Descrição
Problema	Quando o aluno erra uma questão, ele precisa entender o raciocínio correto, mas nem sempre há explicação disponível no banco de dados.
Solução com IA	Ao errar uma questão, o aluno pode solicitar uma explicação detalhada via GPT-4o. A IA recebe o enunciado, as opções, a resposta do aluno e o gabarito, e gera uma explicação didática passo a passo.
API utilizada	OpenAI GPT-4o via API REST. Escolhida pela qualidade pedagógica das explicações e capacidade de raciocínio matemático/científico.
Endpoint	POST /ai/explain — recebe question_id e a resposta do aluno, retorna explicação em texto rico.

```
// services/ai-explain.service.ts
```

```

async function explainQuestion(question: Question, chosenAnswer: string) {
  const prompt = `
Voce e um professor especialista em vestibulares.
Explique de forma didatica e clara por que a alternativa correta esta certa
e por que a alternativa escolhida pelo aluno esta errada.

QUESTAO: ${question.statement}
ALTERNATIVAS:
A) ${question.option_a}
B) ${question.option_b}
C) ${question.option_c}
D) ${question.option_d}

RESPOSTA DO ALUNO: ${chosenAnswer.toUpperCase()}
GABARITO CORRETO: ${question.answer.toUpperCase()}

Estruture sua resposta com:
1. Por que a alternativa correta (${question.answer.toUpperCase()}) esta certa
2. Por que a alternativa ${chosenAnswer.toUpperCase()} esta errada
3. Dica para nao errar esse tipo de questao novamente`;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [{ role: 'user', content: prompt }],
    max_tokens: 500,
    temperature: 0.2,
  });

  return { explanation: response.choices[0].message.content };
}

```

4.3 Funcionalidade 3 — Geração Automática de Flashcards

Atributo	Descrição
Problema	Criar flashcards manualmente é trabalhoso. O aluno precisa de cards de qualidade sem gastar tempo de estudo criando-os.
Solução com IA	O aluno informa um tópico (ex: 'Mitose — Biologia') e a IA gera automaticamente entre 5 e 10 flashcards no formato pergunta/resposta, prontos para serem adicionados ao deck de revisão.
API utilizada	OpenAI GPT-4o-mini. Resposta em JSON estruturado para inserção direta na coleção MongoDB de flashcards.

```

// services/ai-flashcard.service.ts
async function generateFlashcards(subject: string, topic: string, count = 5) {
  const prompt = `
Crie ${count} flashcards de estudo sobre o topico "${topic}" de ${subject}
para alunos do vestibular brasileiro.

Retorne APENAS um JSON com a estrutura:
{

```

```

    "flashcards": [
      { "front": "Pergunta clara e objetiva",
        "back": "Resposta concisa mas completa",
        "tags": ["tag1", "tag2"] }
    ]
  }

  Os cards devem:
  - Ter perguntas diretas e objetivas
  - Ter respostas completas mas sucintas (max 2 linhas)
  - Cobrir conceitos fundamentais do topico para o vestibular`;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [{ role: 'user', content: prompt }],
    response_format: { type: 'json_object' },
    temperature: 0.4,
  });

  const { flashcards } = JSON.parse(response.choices[0].message.content);

  // Inserir diretamente no MongoDB
  await FlashcardRepository.insertMany(
    flashcards.map(fc => ({ ...fc, subject, topic, user_id: 'system' }))
  );

  return flashcards;
}

```

4.4 Justificativa das Ferramentas de IA

Ferramenta	Uso no StudyPath	Justificativa
OpenAI GPT-4o	Explicação de questões erradas	Melhor capacidade de raciocínio pedagógico e matemático. Ideal para explicações detalhadas.
OpenAI GPT-4o-mini	Recomendações e geração de flashcards	Custo-benefício ideal para chamadas frequentes. Qualidade suficiente para JSON estruturado.
Algoritmo SM-2	Repetição espaçada dos flashcards	Algoritmo científico consolidado, sem custo de API, totalmente controlado internamente.

5. CHECKPOINT 2 — ESTADO ATUAL DO PROJETO

5.1 Progresso Geral

Bloco	Entrega	Status	Conclusão
Bloco 1	Concepção — Problema, BMC, Requisitos, User Stories, GitHub	Concluído	100%
Bloco 2	Design & UX — Personas, Wireframes, Figma, Teste de usabilidade	Concluído	100%
Bloco 3	Arquitetura — C4, SQL, NoSQL, Diagrams as Code	Concluído	100%
Bloco 4	Desenvolvimento — GoF, APIs, IA, Checkpoint 2	Em entrega	100%
Bloco 5	Testes, Deploy e Apresentação Final	Pendente	0%

5.2 O Que Foi Concluído

- Definição completa do problema, público-alvo e modelo de negócio SaaS (Business Model Canvas)
- 12 Requisitos Funcionais e 5 Requisitos Não Funcionais documentados
- 12 User Stories e quadro Kanban organizado no GitHub Projects
- 2 Personas detalhadas (Lucas Mendes e Ana Carolina) com mapas de jornada
- 5 Wireframes de baixa fidelidade e 10 telas em alta fidelidade no Figma
- Teste de usabilidade com 2 participantes externos e ajustes documentados
- Diagramas C4 completos (Context, Container, Component) gerados com Mermaid
- Script SQL com 8 tabelas (MySQL 8.0) com PKs, FKs e índices
- Modelagem NoSQL: coleção MongoDB (flashcards + SM-2) e 4 estruturas Redis
- 4 padrões GoF documentados com código TypeScript (Factory, Strategy, Observer, Decorator)
- 21 endpoints documentados em formato OpenAPI com exemplos de request/response
- 3 funcionalidades de IA especificadas e implementadas em PoC (recomendação, explicação, geração de flashcards)

5.3 Em Andamento / Próximos Passos

- Desenvolvimento do backend — Study Service (Node.js + Express + Prisma) — 40% concluído
- Desenvolvimento do frontend — Web App (React + Vite) — 30% concluído
- Configuração do ambiente de produção na AWS (EC2, RDS, ElastiCache, MongoDB Atlas)
- Implementação do fluxo de pagamento com Stripe (webhook de confirmação)
- Testes unitários e de integração com Jest — a iniciar no Bloco 5

5.4 Links do Projeto

Recurso	Link
Repositório GitHub	https://github.com/mindrodr/studypath-vestibular
Quadro Kanban	https://github.com/mindrodr/studypath-vestibular/projects/1
Protótipo Figma	https://www.figma.com/proto/PuPz8UZcRYo1LF3Zlv0qGm/StudyPath?node-id=0-1&t=-jAqitq9TliY2q19a-1
Documentação API	https://github.com/mindrodr/studypath-vestibular/blob/main/docs/api/openapi.yaml
Script SQL	https://github.com/mindrodr/studypath-vestibular/blob/main/docs/database/studypath_schema.sql
Diagramas Mermaid (C4)	https://github.com/mindrodr/studypath-vestibular/tree/main/docs/architecture

5.5 Atualização do Kanban — GitHub Projects

A FAZER	EM PROGRESSO	CONCLUÍDO
• Testes unitários (Jest) • Deploy AWS prod • Webhook Stripe • App Mobile • CI/CD Pipeline	• Backend Study Service • Frontend Web App • Integração OpenAI API • Config AWS (RDS, Redis) • Script de seed do banco	• Documentação (Blocos 1-4) • Diagramas C4 (Mermaid) • Schema SQL + NoSQL • Protótipo Figma • GoF + APIs • PoC de IA